

From zero to your first real project — C#, ASP.NET Core, Entity Framework Core, SQL Server, and Postman, explained end to end.

Table of Contents

- 01 What is .NET (the big picture)
- 02 Setting up your environment (Visual Studio, SQL Server, Postman)
- 03 C# Language Essentials for Backend Work
- 04 Collections in C# (List, Dictionary, IEnumerable, LINQ)
- 05 Project Types: Console vs MVC vs Web API vs Minimal API
- 06 ASP.NET Core Fundamentals (how a request flows through your app)
- 07 Controllers, Routing, and Models
- 08 Dependency Injection (DI) — the heart of .NET
- 09 Entity Framework Core (EF Core) — full explanation
- 10 Connecting to SQL Server & Migrations
- 11 Building a Real CRUD API — Step by Step
- 12 Testing your API with Postman
- 13 DTOs, AutoMapper, and Validation
- 14 Middleware & the Request Pipeline
- 15 Authentication & Authorization (JWT)
- 16 Configuration & appsettings.json
- 17 Logging & Error Handling
- 18 Project Structure (Clean Architecture basics)
- 19 Git, Deployment, and Next Steps
- 20 Suggested Learning Roadmap (Order to Actually Do Things)

1. What is .NET (the big picture)

.NET is a free, open-source developer platform (by Microsoft) for building many types of applications: web APIs, web apps, desktop apps, mobile, cloud services, games, IoT.

Key terms you'll hear:

Term	Meaning
.NET (Core)	The modern, cross-platform version (.NET 6, 7, 8, 9...). This is what you use today.
.NET Framework	The old Windows-only version (4.x). Legacy, still used in old enterprise systems.
C#	The programming language you write .NET code in.
CLR (Common Language Runtime)	The engine that runs your compiled C# code (handles memory, garbage collection, JIT compilation).
SDK	The tools you install to build .NET apps (compiler, CLI, project templates).
NuGet	.NET's package manager (like npm for JavaScript). You install libraries via NuGet.
ASP.NET Core	The framework built on top of .NET specifically for building web apps and APIs.
Entity Framework Core (EF Core)	The ORM (Object-Relational Mapper) that lets you talk to a database using C# objects instead of raw SQL.

Mental model:

```
.NET (platform)
├─ ASP.NET Core (web framework, runs on .NET)
│   ├─ MVC / Web API / Minimal API (patterns for building web apps within ASP.NET Core)
│   │   └─ Entity Framework Core (talks to the database)
│   │       └─ SQL Server (the actual database)
```

You write C# code → ASP.NET Core handles HTTP requests → Controllers process them → EF Core talks to SQL Server → response goes back as JSON → Postman (or a frontend) consumes it.

2. Setting up your environment

You already have the right tools:

- **Visual Studio** — your IDE (Community edition is free). Make sure the **“ASP.NET and web development”** workload is installed (Visual Studio Installer → Modify → check that box).
- **SQL Server Management Studio (SSMS)** — for viewing/managing your database visually.
- **Postman** — for manually testing your API endpoints (sending GET/POST/PUT/DELETE requests).

Also install (usually bundled with Visual Studio): - **.NET SDK** (latest LTS version, e.g. .NET 8) - **SQL Server LocalDB or SQL Server Express** — a lightweight local SQL Server instance for development (Visual Studio installs LocalDB by default)

To check your .NET version, open a terminal:

```
dotnet --version
```

3. C# Language Essentials for Backend Work

You don't need to master everything, but you must be comfortable with:

| Variables & Types

```
int age = 25;
string name = "Ahmed";
bool isActive = true;
double price = 19.99;
DateTime createdAt = DateTime.Now;
```

| Classes & Objects (C# is Object-Oriented)

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

This is a **model/entity** — you'll use classes like this constantly to represent database tables, API requests, and responses.

| Properties (`get; set;`)

These are auto-generated getters/setters. `public string Name { get; set; }` means other code can read and write `Name` safely.

| Access Modifiers

- `public` — accessible from anywhere
- `private` — only inside the same class
- `protected` — class + subclasses
- `internal` — same project only

| `async/await` (CRITICAL for backend work)

Almost everything in ASP.NET Core involving the database or network is asynchronous:

```
public async Task<Product> GetProductAsync(int id)
{
    return await _context.Products.FindAsync(id);
}
```

`async / await` lets your server handle many requests at once without blocking threads while waiting for I/O (database calls, file reads, HTTP calls).

| Interfaces

```
public interface IProductService
{
    Task<List<Product>> GetAllAsync();
}
```

Interfaces define a “contract” — a class that implements it must provide those methods. This is essential for Dependency Injection (explained later).

| LINQ (Language Integrated Query)

LINQ lets you query collections (and databases, via EF Core) using C# syntax instead of loops:

```
var expensiveProducts = products.Where(p => p.Price > 100).ToList();  
var names = products.Select(p => p.Name).ToList();  
var first = products.FirstOrDefault(p => p.Id == 5);
```

4. Collections in C

Collections store groups of objects. You'll use these constantly.

Collection	Description	Example
Array	Fixed size, same type	<code>int[] nums = {1,2,3};</code>
List	Dynamic-size, most commonly used	<code>List<Product> products = new();</code>
Dictionary<TKey,TValue>	Key-value pairs, fast lookup	<code>Dictionary<int,string> map = new();</code>
IEnumerable	Read-only, forward-only sequence (base interface for most collections)	Used as a return type for queries
ICollection	IEnumerable + Add/Remove/Count	Used in EF Core navigation properties
Queue	FIFO (first in, first out)	Task queues
Stack	LIFO (last in, first out)	Undo operations
HashSet	Unique unordered items	Removing duplicates fast

Common operations

```
List<Product> products = new List<Product>();
products.Add(new Product { Id = 1, Name = "Laptop" });
products.Remove(products[0]);
bool exists = products.Any(p => p.Name == "Laptop");
int count = products.Count;

Dictionary<int, string> cities = new Dictionary<int, string>();
cities.Add(1, "Cairo");
string city = cities[1];
bool hasKey = cities.ContainsKey(1);
```

Why this matters for backend work: EF Core returns `List<T>` or `IEnumerable<T>` from database queries, API responses are usually `List<T>` serialized to JSON, and you'll constantly filter/map/sort these using LINQ.

5. Project Types: Console vs MVC vs Web API vs Minimal API

When you create a new project in Visual Studio, you'll see these templates:

- **Console App** — simple, no web server. Good for learning/testing logic.
- **ASP.NET Core Web App (MVC)** — Model-View-Controller. Used for apps that render HTML pages on the server (Views using Razor syntax `.cshtml`).
- **ASP.NET Core Web API** — returns JSON/XML, no HTML views. This is what powers mobile apps, SPAs (React/Angular/Vue), and is what most **backend-only** developers build. **This is likely what you want.**
- **Minimal API** — a newer, lighter way to write Web APIs with less boilerplate (endpoints defined directly in `Program.cs` instead of separate Controller classes).

| MVC Pattern Explained

- **Model** — represents your data (e.g., `Product` class, and classes describing what a request/response looks like)
- **View** — the HTML/UI (only relevant if you're doing server-rendered pages, not needed for pure APIs)
- **Controller** — receives HTTP requests, talks to services/database, returns a response

For a **backend/API-only** project, you'll use **Controllers + Models**, but skip **Views** (since you return JSON, not HTML).

6. ASP.NET Core Fundamentals

When your app starts, `Program.cs` is the entry point. In modern .NET (6+), it looks like this:

```
var builder = WebApplication.CreateBuilder(args);

// Register services (Dependency Injection container)
builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline (middleware)
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

What happens on every request: 1. A request hits your server (e.g., `GET /api/products/5`) 2. It passes through the **middleware pipeline** (logging, authentication, error handling, etc.) 3. The **router** matches the URL to a **Controller action** 4. The controller runs your logic (often calling a service/EF Core) 5. A response (usually JSON) is returned back through the pipeline to the client

7. Controllers, Routing, and Models

A basic Controller

```
[ApiController]
[Route("api/[controller]")] // becomes "api/products"
public class ProductsController : ControllerBase
{
    private readonly AppDbContext _context;

    public ProductsController(AppDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<ActionResult<List<Product>>> GetAll()
    {
        return await _context.Products.ToListAsync();
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Product>> GetById(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product == null) return NotFound();
        return product;
    }

    [HttpPost]
    public async Task<ActionResult<Product>> Create(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(GetById), new { id = product.Id }, product);
    }

    [HttpPut("{id}")]
    public async Task<IActionResult> Update(int id, Product product)
    {
        if (id != product.Id) return BadRequest();
        _context.Entry(product).State = EntityState.Modified;
        await _context.SaveChangesAsync();
        return NoContent();
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product == null) return NotFound();
        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
        return NoContent();
    }
}
```

This single controller gives you a full **CRUD API** (Create, Read, Update, Delete) for a `Product` resource.

HTTP verbs → meaning

Verb	Purpose
GET	Read data
POST	Create new data
PUT	Update/replace existing data

PATCH	Partially update data
DELETE	Remove data

| Routing

`[Route("api/[controller]")] + [HttpGet("{id}")]` means: `GET /api/products/5` calls `GetById(5)`.

8. Dependency Injection (DI)

DI is one of the **most important concepts** in ASP.NET Core. Instead of a class creating its own dependencies, they're "injected" from outside.

| Without DI (bad):

```
public class ProductsController
{
    private ApplicationDbContext _context = new ApplicationDbContext(); // tightly coupled, hard to test
}
```

| With DI (correct .NET way):

```
public class ProductsController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public ProductsController(ApplicationDbContext context) // injected automatically
    {
        _context = context;
    }
}
```

You register services in `Program.cs` :

```
builder.Services.AddDbContext<ApplicationDbContext>(...);
builder.Services.AddScoped<IProductService, ProductService>();
```

| Service Lifetimes

Lifetime	Meaning
<code>AddTransient</code>	New instance every time it's requested
<code>AddScoped</code>	One instance per HTTP request (most common for DbContext, services)
<code>AddSingleton</code>	One instance for the entire application lifetime

Why it matters: DI makes your code testable (you can swap real services for fake/mock ones in tests) and loosely coupled.

9. Entity Framework Core (EF Core) — Full Explanation

EF Core is an ORM (Object-Relational Mapper). It lets you interact with a database using C# classes and LINQ instead of writing raw SQL.

Core Concepts

1. Entity — a C# class representing a database table.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }
    public Category Category { get; set; } // navigation property (relationship)
}

public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<Product> Products { get; set; } = new();
}
```

2. DbContext — represents a session with the database; contains `DbSet<T>` for each table.

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
}
```

3. DbSet — represents a table; you query/add/remove through it (`_context.Products`).

4. Migrations — version-controlled changes to your database schema, generated from your C# model changes.

EF Core Workflow (“Code-First” — most common)

1. Define your C# entity classes (models)
2. Define your `DbContext` with `DbSet<T>` properties
3. Configure the connection string
4. Run a migration to generate SQL that creates/updates tables
5. Apply the migration to update the actual database
6. Query/update data using LINQ through `_context`

Common EF Core Commands (run in Visual Studio’s Package Manager Console, or terminal with `dotnet ef`)

```
# Install EF Core tools (once)
dotnet tool install --global dotnet-ef

# Add EF Core packages to your project
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools

# Create a migration
dotnet ef migrations add InitialCreate

# Apply migration to the database (creates/updates tables)
dotnet ef database update

# Remove last migration (if not yet applied)
```

```
dotnet ef migrations remove
```

| Querying with EF Core (LINQ → translated to SQL automatically)

```
// Get all
var products = await _context.Products.ToListAsync();

// Filter
var cheap = await _context.Products.Where(p => p.Price < 50).ToListAsync();

// Include related data (JOIN)
var productsWithCategory = await _context.Products
    .Include(p => p.Category)
    .ToListAsync();

// Single record
var product = await _context.Products.FirstOrDefaultAsync(p => p.Id == 1);

// Add
_context.Products.Add(newProduct);
await _context.SaveChangesAsync();

// Update
existingProduct.Price = 99.99m;
await _context.SaveChangesAsync();

// Delete
_context.Products.Remove(existingProduct);
await _context.SaveChangesAsync();
```

Important: Nothing is actually sent to the database until you call `SaveChangesAsync()` (except queries, which execute immediately).

| Relationships in EF Core

- **One-to-Many:** Category → Products (one category has many products) — shown above
 - **Many-to-Many:** EF Core can auto-generate a join table if you define collection navigation properties on both sides
 - **One-to-One:** Configured via `HasOne().WithOne()` in `OnModelCreating`
-

10. Connecting to SQL Server & Migrations

| Connection String (in `appsettings.json`)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=
(localdb)\\mssqllocaldb;Database=MyAppDb;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

If using full SQL Server (not LocalDB), it might look like:

```
"Server=localhost;Database=MyAppDb;User Id=sa;Password=YourPassword;TrustServerCertificate=True"
```

| Registering it in `Program.cs`

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

| Viewing your database in SSMS

1. Open SQL Server Management Studio
 2. Connect to `(localdb)\\mssqllocaldb` (or your server name)
 3. Expand Databases → you should see your database and tables after running `dotnet ef database update`
 4. You can run raw SQL here too, to inspect/debug data directly
-

11. Building a Real CRUD API — Step by Step

Here's the exact sequence for starting a real project from scratch:

1. **Create the project** in Visual Studio: `File → New → Project → ASP.NET Core Web API`
 2. **Name it** (e.g., `ShopApi`), choose .NET 8, uncheck "Use controllers" only if you want Minimal API — otherwise leave it checked
 3. **Install NuGet packages:** `Microsoft.EntityFrameworkCore.SqlServer`, `Microsoft.EntityFrameworkCore.Tools`, `Microsoft.EntityFrameworkCore.Design`
 4. **Create your Models** folder → add entity classes (`Product.cs`, `Category.cs`, etc.)
 5. **Create Data folder** → add `AppDbContext.cs`
 6. **Add connection string** to `appsettings.json`
 7. **Register DbContext** in `Program.cs`
 8. **Run migrations:** `dotnet ef migrations add InitialCreate` then `dotnet ef database update`
 9. **Create Controllers** folder → add a controller per resource (`ProductsController.cs`)
 10. **Write CRUD endpoints** (GET, POST, PUT, DELETE) as shown in Section 7
 11. **Run the project** (F5 in Visual Studio) — it opens Swagger UI at `https://localhost:xxxx/swagger` automatically (built-in API testing page)
 12. **Test with Postman** (see next section)
 13. **Add business logic layer** (Services) — don't put everything in controllers
 14. **Add DTOs and validation**
 15. **Add authentication** if needed
 16. **Add logging & error handling**
 17. **Deploy**
-

12. Testing your API with Postman

Once your app is running (e.g., `https://localhost:7101`):

1. Open Postman → create a **New Request**
2. Set method to `GET`, URL: `https://localhost:7101/api/products`
3. Click **Send** — you should see a JSON array response
4. For `POST`, set method to `POST`, same base URL + `/api/products`, go to **Body** → **raw** → **JSON**, and enter:

```
{
  "name": "Keyboard",
  "price": 25.99,
  "categoryId": 1
}
```

5. Click **Send** — you should get a `201 Created` response with the new object
6. For `PUT` / `DELETE`, use the URL with the ID: `.../api/products/1`

Tip: Group related requests into a **Postman Collection**, and use **Environment Variables** (e.g., `{{baseUrl}}`) so you can switch between local/staging/production easily.

Bonus: ASP.NET Core auto-generates a Swagger/OpenAPI page (`/swagger`) — you can test endpoints directly in the browser too, without Postman, which is great for quick checks.

13. DTOs, AutoMapper, and Validation

DTO = Data Transfer Object. Never expose your database entities directly to the outside world — use separate classes for input/output. This avoids leaking internal fields and lets you shape the API response independently from your database schema.

```
public class ProductDto
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}

public class CreateProductDto
{
    [Required]
    [MaxLength(100)]
    public string Name { get; set; }

    [Range(0.01, 100000)]
    public decimal Price { get; set; }
}
```

| Validation

`[ApiController]` automatically validates data annotations (`[Required]` , `[Range]` , etc.) and returns a `400 Bad Request` automatically if invalid — you don't need to check `ModelState.IsValid` manually in most cases, but you can:

```
if (!ModelState.IsValid) return BadRequest(ModelState);
```

| AutoMapper (optional but common)

A library that maps between entities and DTOs automatically instead of writing manual mapping code:

```
var dto = _mapper.Map<ProductDto>(product);
```

14. Middleware & the Request Pipeline

Middleware are components that process every request/response in sequence. Order matters!

```
app.UseExceptionHandler("/error"); // catch unhandled exceptions
app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthentication();           // who are you?
app.UseAuthorization();           // are you allowed?
app.MapControllers();
```

You can also write **custom middleware** (e.g., for logging every request):

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"Request: {context.Request.Method} {context.Request.Path}");
    await next.Invoke();
});
```

15. Authentication & Authorization (JWT)

Most real APIs need to know **who** is calling them (Authentication) and **what** they're allowed to do (Authorization).

JWT (JSON Web Token) is the standard approach for APIs:

1. User logs in with username/password → server verifies credentials
2. Server generates a signed JWT token containing user info (claims)
3. Client stores the token and sends it in the `Authorization: Bearer <token>` header on every future request
4. Server validates the token on each request (no database lookup needed — it's self-contained)

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = config["Jwt:Issuer"],
            ValidAudience = config["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(config["Jwt:Key"]))
        };
    });
```

Protect an endpoint:

```
[Authorize]
[HttpGet("secret")]
public IActionResult GetSecret() => Ok("Only for logged-in users");

[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult DeleteAdmin(int id) => ...
```

16. Configuration & appsettings.json

`appsettings.json` holds settings like connection strings, JWT keys, and any custom config, separated per environment (`appsettings.Development.json` , `appsettings.Production.json`).

Access it via `IConfiguration` :

```
var value = builder.Configuration["Jwt:Key"];
var connStr = builder.Configuration.GetConnectionString("DefaultConnection");
```

Never commit real secrets (passwords, API keys) to Git — use **User Secrets** in development (`dotnet user-secrets`) and environment variables/Azure Key Vault in production.

17. Logging & Error Handling

Built-in logging:

```
public class ProductsController : ControllerBase
{
    private readonly ILogger<ProductsController> _logger;

    public ProductsController(ILogger<ProductsController> logger)
    {
        _logger = logger;
    }

    [HttpGet]
    public IActionResult Get()
    {
        _logger.LogInformation("Fetching all products");
        return Ok();
    }
}
```

| Global exception handling

Instead of try/catch in every controller, use middleware to catch all unhandled exceptions in one place and return a consistent error response (e.g., a `ProblemDetails` JSON object with status code, message, and trace id).

18. Project Structure (Clean Architecture basics)

A simple, scalable folder structure for a real project:

```
ShopApi/  
├─ Controllers/      → API endpoints  
├─ Models/           → Entities (database tables)  
├─ DTOs/             → Request/response shapes  
├─ Data/  
│   └─ AppDbContext.cs  
├─ Services/         → Business logic (interfaces + implementations)  
│   ├── IProductService.cs  
│   └─ ProductService.cs  
├─ Repositories/     → (optional) data access abstraction layer  
├─ Migrations/       → auto-generated by EF Core  
├─ Middleware/       → custom middleware  
├─ appsettings.json  
└─ Program.cs
```

Layered approach: Controller → Service (business logic) → Repository/DbContext (data access). This keeps controllers thin and logic testable.

19. Git, Deployment, and Next Steps

- Use **Git** from day one (`git init` , commit often, push to GitHub). Add a `.gitignore` for .NET (Visual Studio generates one).
 - Never commit `bin/` , `obj/` , or real secrets.
 - Deployment options: **Azure App Service**, **AWS**, **Docker containers**, or a VPS with IIS/Kestrel + Nginx reverse proxy.
 - For a portfolio project, Azure App Service (free tier) + Azure SQL Database is the fastest way to get something live.
-

20. Suggested Learning Roadmap (Do things in this order)

1. ✓ Get comfortable with C# basics: variables, classes, loops, LINQ, async/await
2. ✓ Master collections (List, Dictionary) and LINQ queries
3. ✓ Build a Console app that manipulates a `List<Product>` in memory (no database yet) — practice CRUD logic and LINQ
4. ✓ Create your first ASP.NET Core Web API project, build simple GET endpoints returning hardcoded data
5. ✓ Add EF Core + SQL Server, connect to a real database, do migrations
6. ✓ Build full CRUD for one entity (Products) — test every endpoint in Postman
7. ✓ Add a second entity with a relationship (Category → Products), practice `.Include()`
8. ✓ Introduce DTOs and a Service layer — stop putting logic directly in controllers
9. ✓ Add validation and proper error handling
10. ✓ Add JWT authentication with login/register endpoints
11. ✓ Add logging
12. ✓ Refactor into a clean folder structure
13. ✓ Deploy it somewhere live (Azure free tier is easiest)
14. ✓ Build a second, bigger project (e.g., a blog API, e-commerce API, or task manager) applying everything above from scratch, unaided

Final note

The fastest way to actually learn this is: **build one small real CRUD API end-to-end this week** — one entity, full CRUD, connected to SQL Server, tested in Postman — before adding any advanced features. Everything else (auth, DTOs, layers) makes far more sense once you've felt the basic request → controller → EF Core → database → response loop work with your own hands.